

Remote code execution via MIDI messages



How I hacked my music synthesizer and
made it play Bad Apple on its LCD

Anna “porta” Antonenko

[@portasynthinca3](https://psi3.ru)
portasynthinca3@gmail.com

Who am I?

- I develop firmware in C at Flipper Devices
 - All opinions expressed are my own.
Research was done on my own.
- I like the BEAM ecosystem (Erlang and Elixir)
 - Writing an OS in Erlang in my spare time
- I like electronics, both digital and analog
 - ...although my specialty is digital
- Decent forward engineer, not a good reverse engineer
- I like electronic music and play the piano

Demo

- MIDI is for exchanging musical data
- MIDI has very simple commands
 - Play a note
 - Stop a note
 - Change the reverb level
 - etc.
- Apparently, even MIDI can have a backdoor
- [Video: backdoor exploitation demo]

What is this talk about?

- Hardware reverse engineering
 - Dealing with an undocumented chip
 - Accessing debug interfaces
 - Reading schematics
- Firmware reverse engineering
 - Finding the stuff you need
 - Making sense of the code
- Firmware exploitation
- A cool story!

The story

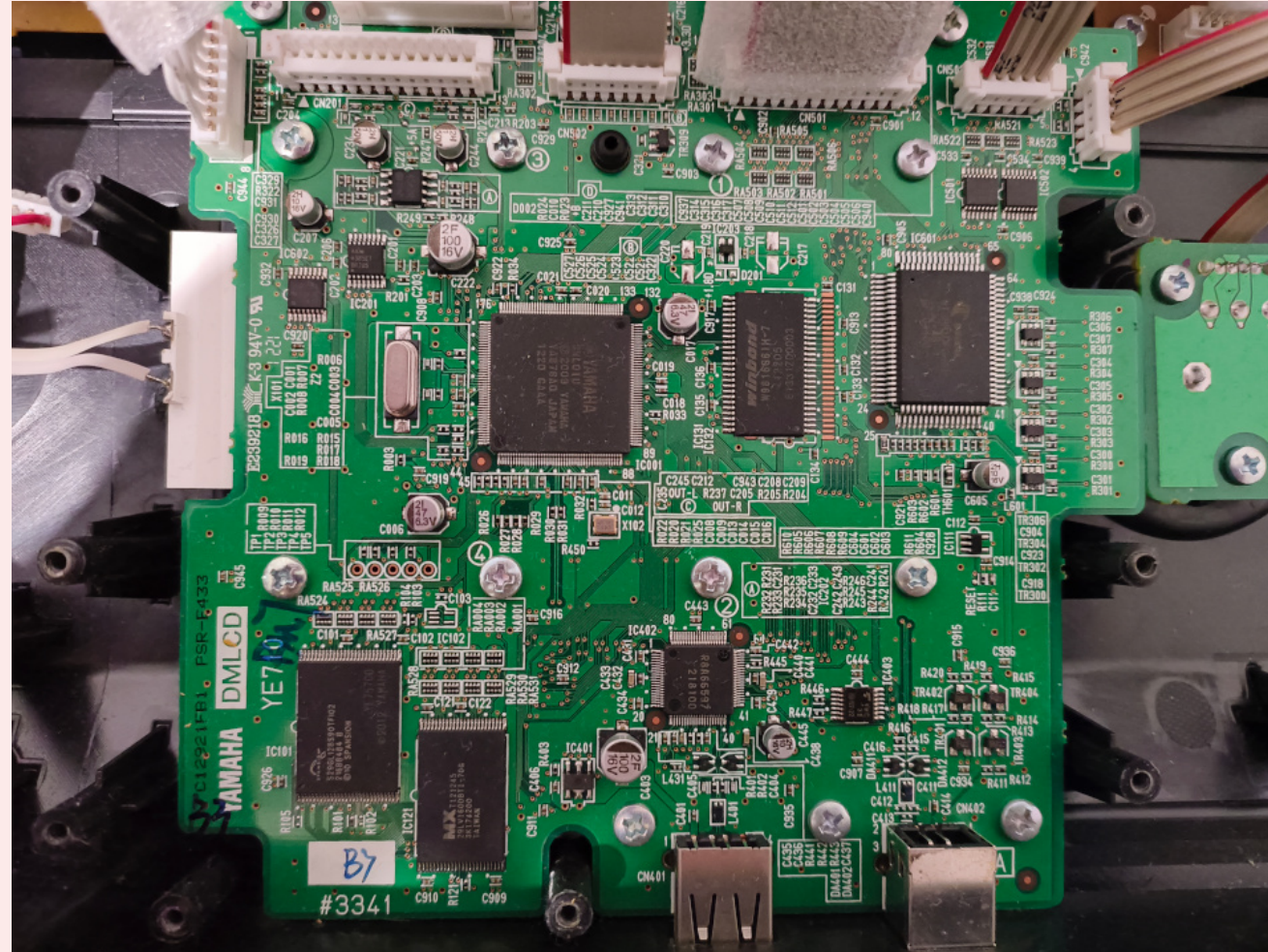


A dirty instrument

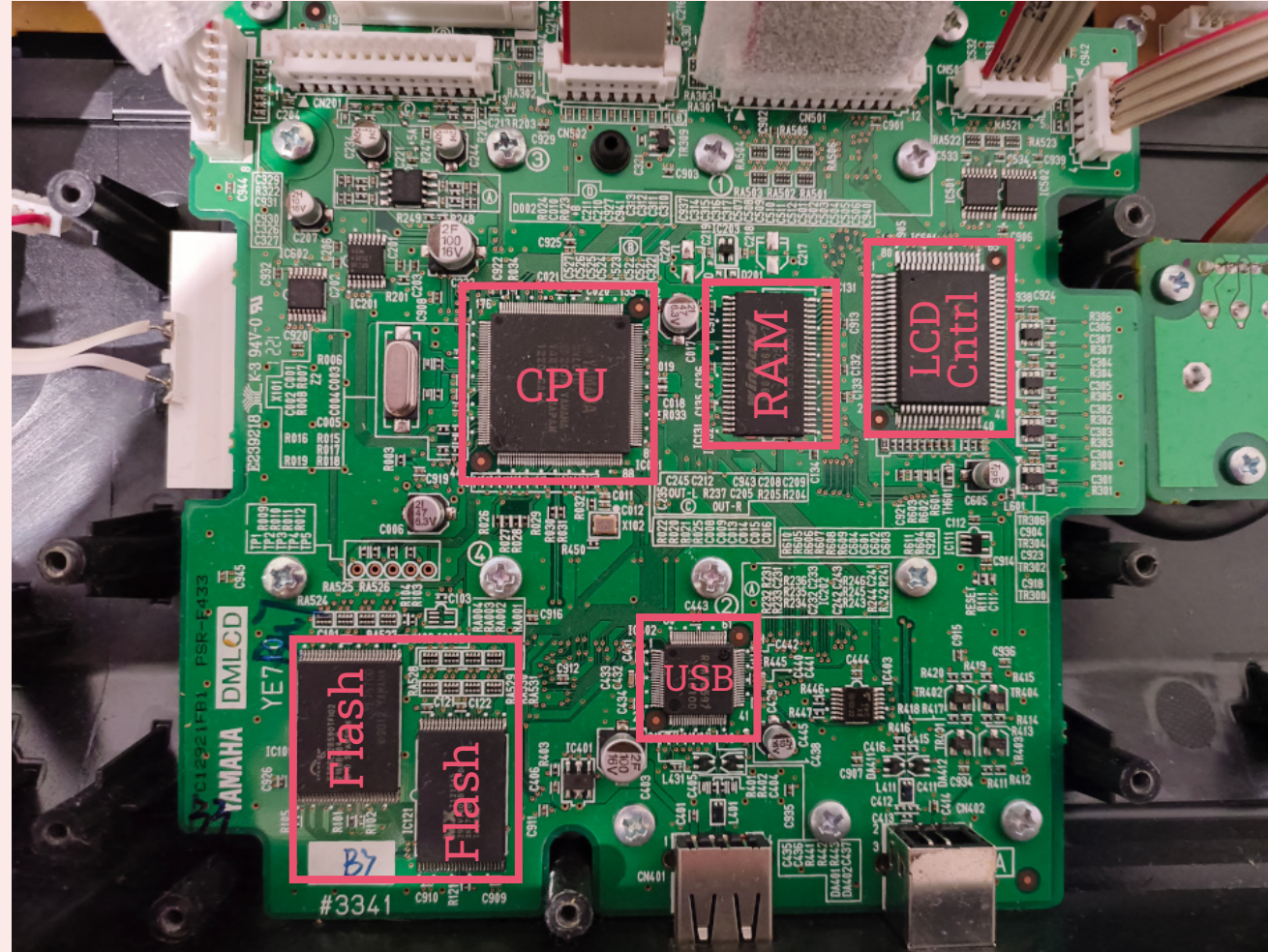
- Had this Yamaha PSR-E433 for a very long time
- It's gotten dusty over the years
- Let's tear it down and clean it
- Find out what makes it tick while we're at it



Main board



Main board



Custom main chip

- Main board features an interesting custom chip: Yamaha SWL01U
- Almost no info about the chip online



Custom main chip

- This article suggests it might be based on the Hitachi SuperH (“SH”) architecture
- The CPU of the Sega Dreamcast was based on SH too
- I have zero experience with SH
- No idea how to approach the chip

“Yamaha <...> have a long history with <...> SH architecture CPUs from Hitachi <...>”

“Yamaha have a considerable investment in software built and tuned for the SH family. Thus, migration to <...> ARM is a pretty big deal <...>”

“Time and experience will show if ARM is adopted in the entry- and mid-range segments, too.”

Paul J. Drongowski
sandsoftwaresound.net/swl-micro-architecture

2 years later



A service manual

- I found this online!
- It's for another model: the E443, not the E433
- The differences are negligible
 - E443 has an audio input, mine doesn't
 - Component numbering is different
 - Same board layout
 - Same main chips (incl. CPU)
 - Same physical appearance



A service manual

- It's got:
 - Full schematic
 - Full board layout
 - Pinouts of the major chips
 - Pinout of the CPU!
- No info about the CPU aside from the pinout
- Still, the pinout and schematic are very helpful

PIN NO.	NAME	I/O	FUNCTION	PIN NO.	NAME	I/O	FUNCTION
1	NC	-	Connection to the GND	88	NC	-	Connection to the GND
2	NC	-		89	NC	-	
3	EAPOK902	I	ON/OFF signal from the keyboard	90	MD03	I/O	
4	EAPOK903	I		91	MD11	I/O	
5	EAPOK904	I		92	MD04	I/O	
6	EAPOK905	I		93	MD12	I/O	
7	EAPOK906	I/O		94	MD05	I/O	
8	EAPOK907	I/O		95	MD13	I/O	External memory data bus
9	EAPOK908	I/O		96	MD06	I/O	
10	EAPOK909	I/O		97	MD14	I/O	
11	EAPOK910	I/O	Key selection signal to the keyboard	98	MD07	I/O	
12	EAPOK911	I/O		99	MD15	I/O	
13	EAPOK912	I/O		100	Vcc	-	Digital Core ground
14	EAPOK913	I/O		101	32B0402	O	Priority of setup P04 > RCU(SDRAM) > CS66F0N
15	PROTIN	I	Determines if the product is a prototype	102	MU79A04	O	
16	BISTMD	I	Memory BIST mode (1: BIST mode)	103	MA23	O	
17	BISTCLK	I	Memory BIST clock	104	MA15	O	
18	PLUPIN	I	PLS input mode (0: PLS input)	105	MA16	O	External memory address
19	TESTIN	I	Test mode	106	MA17	O	
20	Vcc	-	Digital Core ground	107	MA18	O	
21	XI	I	Digital Core input (33.868 MHz)	108	MA19	O	
22	XO	O	Digital Core output	109	MA11	O	Digital Core power supply
23	Vcc	-	Digital Core power supply	110	Vcc	-	I/O power supply
24	Vcc	-	Digital Core ground	111	ICVcc	I/O	
25	ICVcc	I/O	I/O power supply	112	MA12	O	
26	TESTIN	I	JTAG Vp reset	113	MA20	O	
27	TMS	I	JTAG Vp mode	114	MA21	O	
28	TCK	I	JTAG Vp clock	115	MA22	O	
29	TDI	I	JTAG Vp input	116	MA21PFF1	O	
30	TDO	O	JTAG Vp output	117	MA22PFF2	O	
31	Vcc	-	Digital Core power supply	118	MA19	O	
32	PLUPIN	I	Digital PL power supply (common with Core power supply mode)	119	MU79A04	O	External memory address
33	Vcc	-	Digital Core ground	120	MA26	O	
34	PLUPIN	I	Digital PL ground (common with Core ground mode)	121	MA27	O	
35	WDRCLK	I	Watchdog clock (1 Hz = 48.1 kHz)	122	MA28	O	
36	PFO	O	Output-only port	123	MA29	O	
37	SDD	O	Audio output data (with EQ & compression)	124	MA24	O	
38	BDPWRIN	I	BDP input power (with EQ & compression)	125	MA23	O	
39	BDCLK	O	BD clock (54°)	126	MA22	O	
40	BDCLKP0	O	System clock (200MHz/200MHz)	127	MA20	O	
41	SDPWRIN	I	Serial media input power	128	CS10P01	O	Chip select for area 3
42	Vcc	-	Digital Core ground	129	Vcc	-	Digital Core ground
43	BDCLKP0	O	Interrupt input	130	ICN	I	Reset
44	NC	-		131	Vcc	-	Digital Core ground
45	NC	-		132	NC	-	Connection to the GND
46	NC	-		133	NC	-	
47	NC	-		134	NC	-	
48	TXDOPIN4	O	Serial port Vp	135	PA0	I/O	
49	RXDOPIN4	I	Serial port Vp	136	PA1	I/O	
50	TXDOPIN2	O	Serial port Vp	137	PA2	I/O	
51	RXDOPIN2	I	Serial port Vp	138	PA3	I/O	
52	SCCLKP0	I	Serial port Vp	139	PA4	I/O	
53	UCTL	I	Pin/L when USB is in use/fix H when not in use	140	PA5	I/O	
54	Vcc	-	Digital Core power supply	141	PA6	I/O	
55	Vcc	-	Digital Core ground	142	PA7	I/O	
56	ANVcc	-	Analog power supply	143	PA8	I/O	Universal I/O port
57	ANVcc	-	ADC reference	144	PA9	I/O	
58	AND	I		145	PA0	I/O	
59	ANI	I		146	PA1	I/O	
60	AND	I		147	PA2	I/O	
61	ANI	I		148	PA3	I/O	
62	ANDP0	I	ADC ground reference	149	PA4	I/O	
63	ANVcc	-	Analog ground	150	PA5	I/O	
64	USEVcc	-	USB Vp power supply 1.8V (Pullup when not in use)	151	ICVcc	I/O	I/O power supply
65	USEVcc	-	USB data +	152	Vcc	-	Digital Core ground
66	USEVcc	-	USB data -	153	Vcc	-	Digital Core power supply
67	USEVcc	-	USB Vp power supply 2.5V (Pullup when not in use)	154	WDRCLK	O	Priority of setup PFF > WDRCLK > WDRCLK
68	USEVcc	-	USB Vp power supply 2.5V (Pullup when not in use)	155	WDRCLK	O	Priority of setup PFF > WDRCLK > WDRCLK
69	Vcc	-	Digital Core ground	156	WDRCLK	O	Priority of setup PFF > WDRCLK > WDRCLK
70	Vcc	-	Digital Core power supply	157	WDRCLK	O	Chip select for area 8
71	XO	O	Digital Core input (48 MHz)	158	WDRCLK	O	Chip select for area 5
72	XO	O	Digital Core output	159	WDRCLK	O	Chip select for area 6
73	Vcc	-	Digital Core ground	160	WDRCLK	O	Chip select for area 7
74	ICVcc	I/O	I/O power supply	161	WDRCLK	O	
75	VBUS	I	USB Vbus	162	WDRCLK	O	
76	PULLUP	O	USB Pullup enable	163	WDRCLK	O	
77	CS66F0N	O	Chip select for area 2	164	WDRCLK	O	
78	CS66F0N	O	External memory read signal	165	WDRCLK	O	
79	MA01	O	External memory address	166	WDRCLK	O	
80	MD06	I/O		167	WDRCLK	O	
81	MD08	I/O		168	WDRCLK	O	
82	MD01	I/O		169	WDRCLK	O	
83	MD03	I/O		170	WDRCLK	O	
84	MD02	I/O		171	WDRCLK	O	
85	MD10	I/O		172	WDRCLK	O	
86	NC	-		173	WDRCLK	O	
87	NC	-		174	WDRCLK	O	
88	NC	-		175	WDRCLK	O	
89	NC	-		176	WDRCLK	O	

Approaching the chip

- When reversing, look from the perspective of the forward engineer
- How does one debug their program?
 - A debugger
 - Pause
 - Step-through
 - Examine state
 - Modify state
 - Simple text logs
- There are common hardware interfaces for these
 - Debugging: JTAG, SWD, etc.
 - Simple text: UART (also known as a serial port)

Gaining logs access

- If a chip is outputting logs, most of the time it'll be over UART
- UART requires one wire per direction
- The chip has 2 UART interfaces, each has a receive and a transmit pin
- Out of the 4 pins, 3 are not used as UART based on the schematic
- The remaining pin (48) is a transmit pin
- Let's listen to the pin

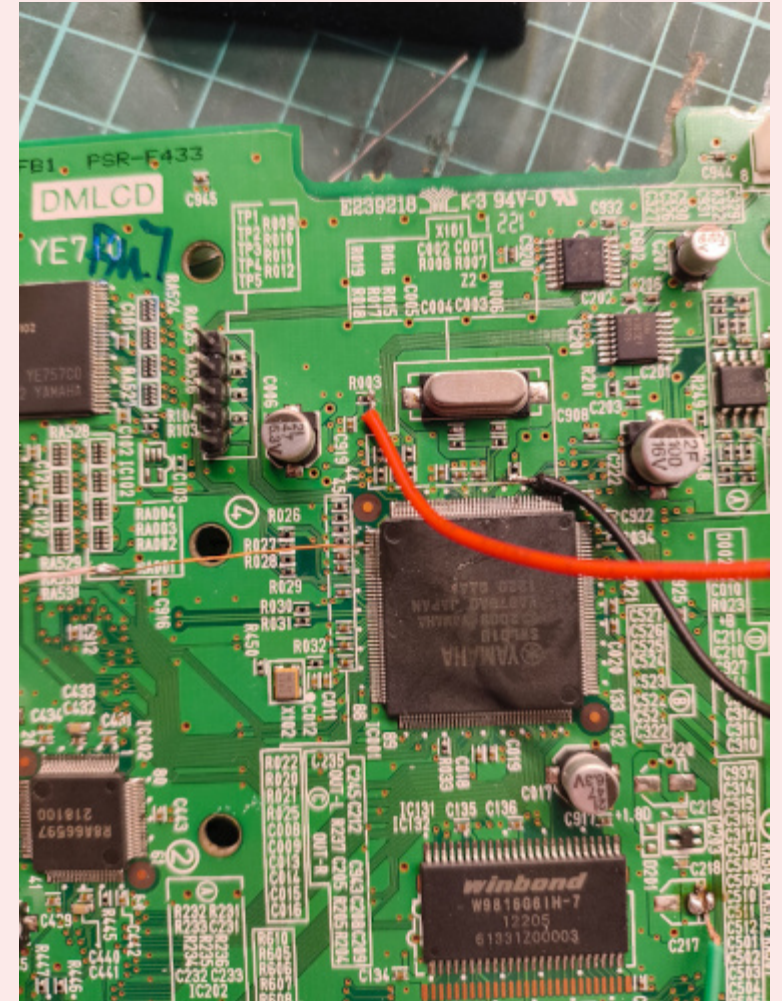
48	TXD0/PG4	O	Serial port I/F
49	RXD0/PH4	I	Serial port I/F
50	TXD1/PG2	O	Serial port I/F
51	RXD1/PH1	I	Serial port I/F
52	SCLK1/PH2	I	Serial port I/F

Gaining logs access

- UART has lots of settings
 - Baud rate (usually from ~300 to ~1M bits/s)
 - Frame length (6 - 8 bits)
 - Stop bits (1, 1.5, or 2)
 - Parity bit (even, odd or none)
 - Way too many combinations of these
- The transmitter never announces its settings, it just yells into the void
- The receiver has to have the same settings
- It's easier to look at the raw signal, infer the settings, then connect an actual receiver

Gaining logs access

- Connected a logic analyzer to the TXD0 pin
- Total silence.



Gaining logs access

- The chip has configuration pins
- PROTN and TESTN sound promising
 - “N” usually means “inverted”
- Let’s try to enable these modes

15	PROTN	I	Determines if the product is a prototype
16	BISTMD	I	Memory BIST mode (1: BIST mode)
17	BISTCLK	I	Memory BIST clock
18	PLLBPN	I	PLL bypass mode (0: PLL bypass)
19	TESTN	I	Test mode

Gaining logs access

- Still, total silence in all 4 combinations of these 2 pins
- If there are logs, it looks like they're not over UART
- Let's move on to debugging then

Gaining logs access

- The chip has these interesting PROTN and TESTN pins
- Total silence on the UART in all configurations
- If there are logs, it looks like they're not over UART

15	PROTN	I	Determines if the product is a prototype
16	BISTMD	I	Memory BIST mode (1: BIST mode)
17	BISTCLK	I	Memory BIST clock
18	PLLBPN	I	PLL bypass mode (0: PLL bypass)
19	TESTN	I	Test mode

Gaining debug access

- The chip has a JTAG interface
- It's very common, but very dumb and abstract
- If you don't have a description of the chip's internals in a special language (BSDL), you won't be able to use JTAG
- Descriptors for common CPU architectures are available, but you have to know the architecture definitively

26	TRSTN	I	JTAG I/F reset
27	TMS	I	JTAG I/F mode
28	TCK	I	JTAG I/F clock
29	TDI	I	JTAG I/F input
30	TDO	O	JTAG I/F output

Gaining debug access

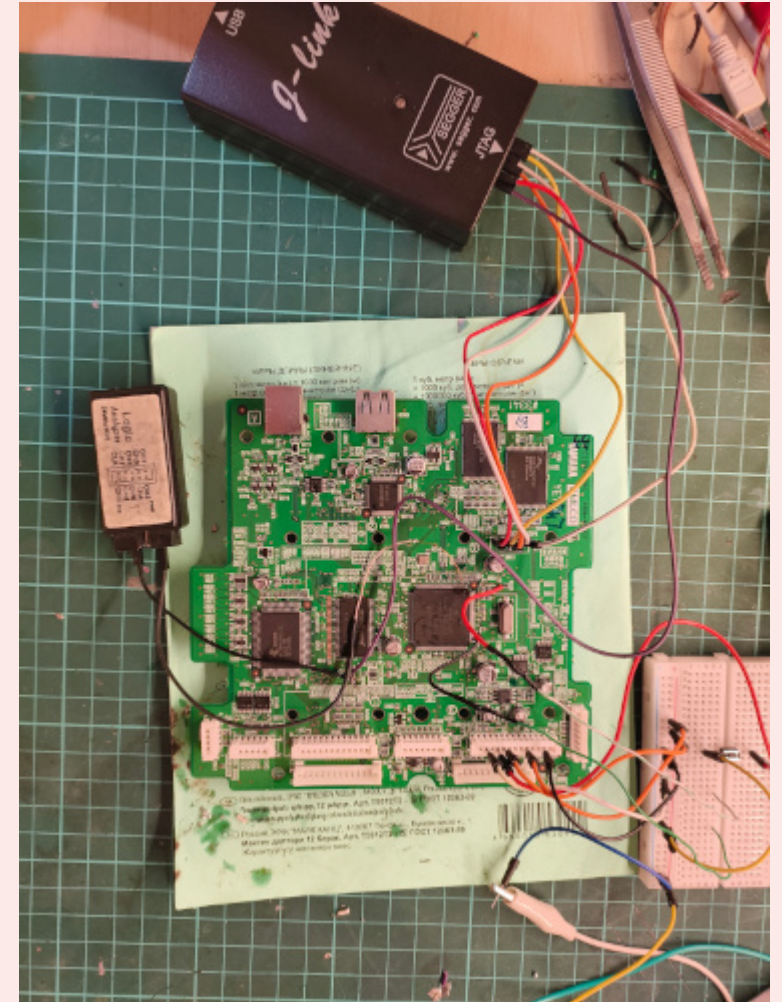
- Talk to the chip incorrectly and you risk releasing the magic smoke
 - Although that risk is tiiiiiiiny
- Almost all JTAG-enabled chips have a self-reported “IDCODE”
 - The specification says it has to be 32-bit
 - No location requirement
 - No presence requirement
- If we acquire the IDCODE, we can search for it online

Gaining debug access

- Hardware tool: JTAG adapter, aka JTAG cable, aka JTAG probe
 - J-Link
 - XJTAG
 - Black Magic
 - FT232H
 - ...lots more
- Software tools:
 - OpenOCD (drives the JTAG adapter)
 - GDB or other GDB server compatible debugger

Gaining debug access

- Connected a JTAG adapter
- Launched OpenOCD
 - It reports the IDCODE by default



Gaining debug access

```
$ cat openocd.cfg
# Uh-oh, a continuity error! I've switched to an FT232R-based dongle instead of J-Link since I
took the picture on the previous slide.
adapter driver ft232r
transport select jtag
adapter speed 5000
$ openocd
Open On-Chip Debugger 0.12.0
Licensed under GNU GPL v2
For bug reports, read
    http://openocd.org/doc/doxygen/bugs.html
Info : only one transport option; autoselect 'jtag'
Warn : Transport "jtag" was already selected
adapter speed: 5000 kHz

Info : Listening on port 6666 for tcl connections
Info : Listening on port 4444 for telnet connections
Info : clock speed 3000 kHz
Warn : There are no enabled taps.  AUTO PROBING MIGHT NOT WORK!!
Info : JTAG tap: auto0.tap tap/device found: 0x3f0f0f0f (mfg: 0x787 (<unknown>), part: 0xf0f0,
ver: 0x3)
Warn : AUTO auto0.tap - use "jtag newtap auto0 tap -irlen 4 -expected-id 0x3f0f0f0f"
Warn : gdb services need one or more targets defined
```

Gaining debug access

- According to OpenOCD, the IDCODE is 0x3f0f0f0f
- Suspiciously pretty
 - Checked multiple times under different conditions to make sure
- Two datasheets mention this IDCODE
 - AT91SAM7SE by Atmel
 - ADuC7000 series by Analog Devices
 - Both are based on ARM7 (not the same as ARMv7)
- Mentions online about other ARM7-based MCUs
- Not sure about the actual architecture
 - Article from before says SH
 - IDCODE says ARM7
- Let's try ARM7

Gaining debug access

```
$ cat openocd.cfg
adapter driver ft232r
transport select jtag
adapter speed 5000
jtag newtap swl01u cpu -irlen 4 -expected-id 0x3f0f0f0f
target create swl01u.cpu arm7tdmi -chain-position swl01u.cpu
$ openocd
Open On-Chip Debugger 0.12.0
Licensed under GNU GPL v2
For bug reports, read
    http://openocd.org/doc/doxygen/bugs.html
Info : only one transport option; autoselect 'jtag'
Warn : Transport "jtag" was already selected
swl01u.cpu
Info : Listening on port 6666 for tcl connections
Info : Listening on port 4444 for telnet connections
Info : clock speed 3000 kHz
Info : JTAG tap: swl01u.cpu tap/device found: 0x3f0f0f0f (mfg: 0x787 (<unknown>), part: 0xf0f0,
ver: 0x3)
Info : Embedded ICE version 1
Info : swl01u.cpu: hardware has 2 breakpoint/watchpoint units
Info : starting gdb server for swl01u.cpu on 3333
Info : Listening on port 3333 for gdb connections
```

Gaining debug access

- Magic smoke hasn't escaped the chip yet
- No OpenOCD errors
- Let's try GDB

~~ed~~ Gaining debug access

- Current draw makes sense:
 - Running: 114 mA
 - Paused: 98 mA
- Interrupt vectors make sense
- Addresses make sense
- It was ARM all along!
- Debug interfaces are usually locked down. We're very lucky!
 - A synth is not a security device

```
(gdb) c          # Continue (unpause)
Continuing.
^C              # Ctrl-C (pause)
Program received signal SIGINT, Interrupt.
0x0206eeb2 in ?? ()

# in ARM7, interrupt vectors start at address 0
# they are regular instructions
(gdb) x/2xw 0 # eXamine 2 heX Words at location 0
0x0: 0xe59ff018 0xe59ff018
(gdb) x/2i 0 # eXamine 2 Instructions at location 0
0x0: ldr pc, [pc, #24] @ 0x20 # jump
0x4: ldr pc, [pc, #24] @ 0x24 # jump

(gdb) where      # address of current instruction
0x02020274 in ?? ()
```

Extracting the firmware



Extracting the firmware

- CPU follows vectors when interrupts arise:
 - CPU is powered up or reset
 - Program did something catastrophic
 - External device requires CPU's attention
- In embedded, the vector table is usually read-only
- ARM7's vector table is at address 0
- The firmware is most likely at address 0
- Flash chip is 16 MiB in size
 - Let's dump 16 MiB from address 0 into a file

Extracting the firmware

- Look at the strings first
 - They are immediately understandable and easily searchable
 - Machine instructions are not
- Our strings definitely make sense
- Success? Not quite
 - Strings repeat every 64 KiB through the entire 16 MiB image
 - All data repeats every 64 KiB
 - Not a lot of strings, actually
- We've captured 256 copies of the same 64 KiB snippet

0x000046a0	T \ #@C#T{\ }#0C
0x000065db	Divide by zero
0x00006ec0	Stack overflow\n
0x0000701b	!Stack overflow
0x000074ba	p:Onx;Oo\8Oe\zB
0x0000756b	IL?R`bILBx L>T`
0x0000bfd0	SWL01U Internal
0x00006a08	Recursive Trap
0x00006c28	Prefetch Abort
0x00000a04	I\n\\n1[\bI\tr\w@
0x00006bc0	Unknown Error
0x0000a83e	êNOPQSTUeqfr
0x00002203	Ix8IHp\bxxpHx
0x00002273	Gx\b>\bp(xHphx

Extracting the firmware

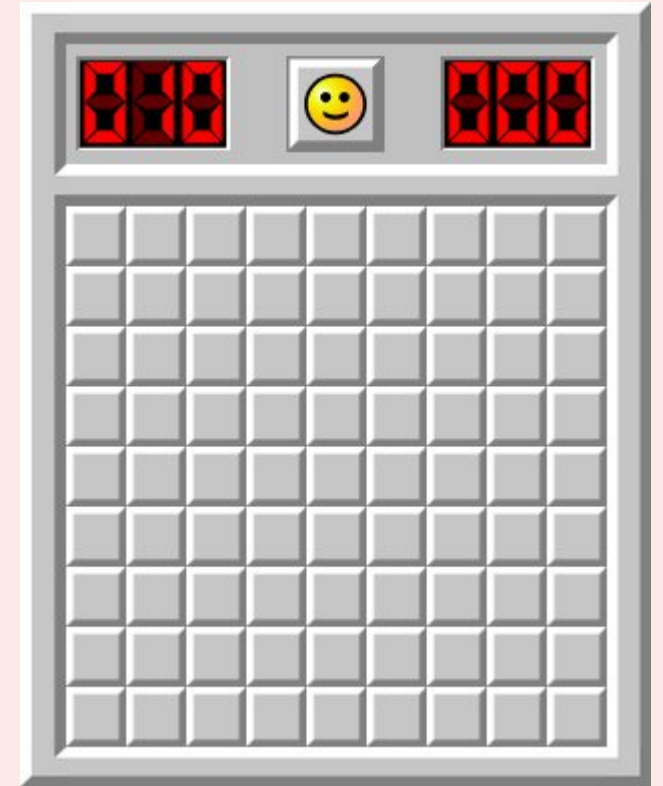
- Reset vector jumps to address 0x02000000
- Data at that address makes even more sense
 - No noticeable repetitions
 - Even more recognizable strings
 - Instrument names
 - UI messages
 - etc.
- Previous image is likely the internal ROM
 - One of its strings says “SWL01U Internal”
 - Repetition caused by simplistic address decoder
- New image is likely the external flash

Playing Minesweeper



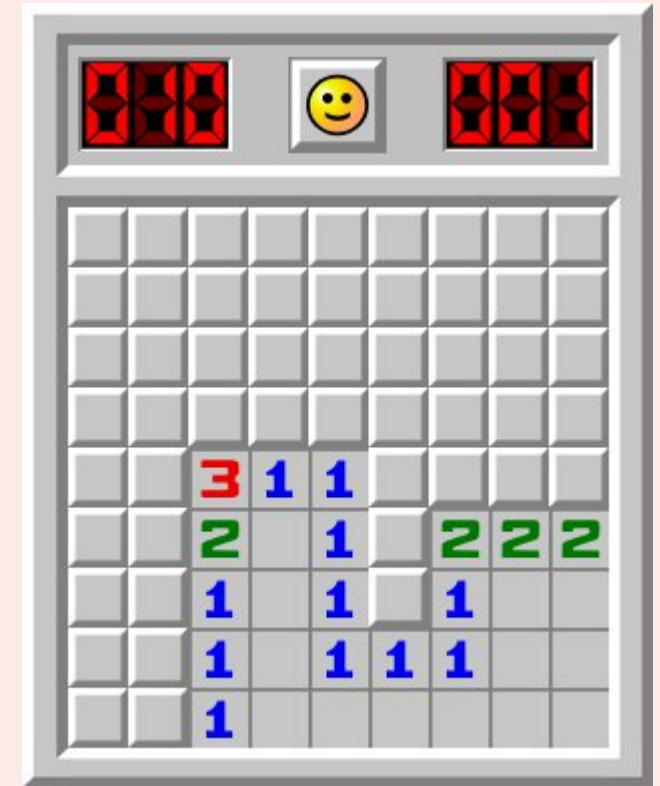
Minesweeper

- Start without knowing the states of any cells



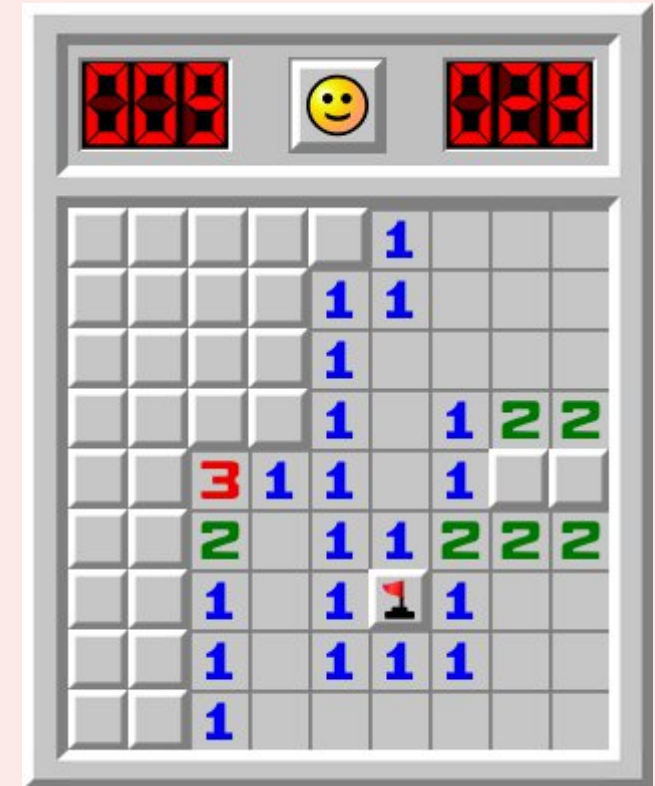
Minesweeper

- Click on a random cell to start the game
- That cell doesn't have a mine (as per game rules)
- Discover the state of neighboring cells



Minesweeper

- All cells tell you something about their neighbors



Minesweeper

- Repeat until you've discovered everything there is to discover on the board



Reverse engineering

- Start without knowing what any of the functions do
- Have an entry point that is understandable as-is
 - Reset vector
 - Strings
- Learn something about functions that are connected with it
 - Learn something about functions that are connected with those ones
 - ...and then those ones
 - ...and then those ones
- Done!

Reverse engineering

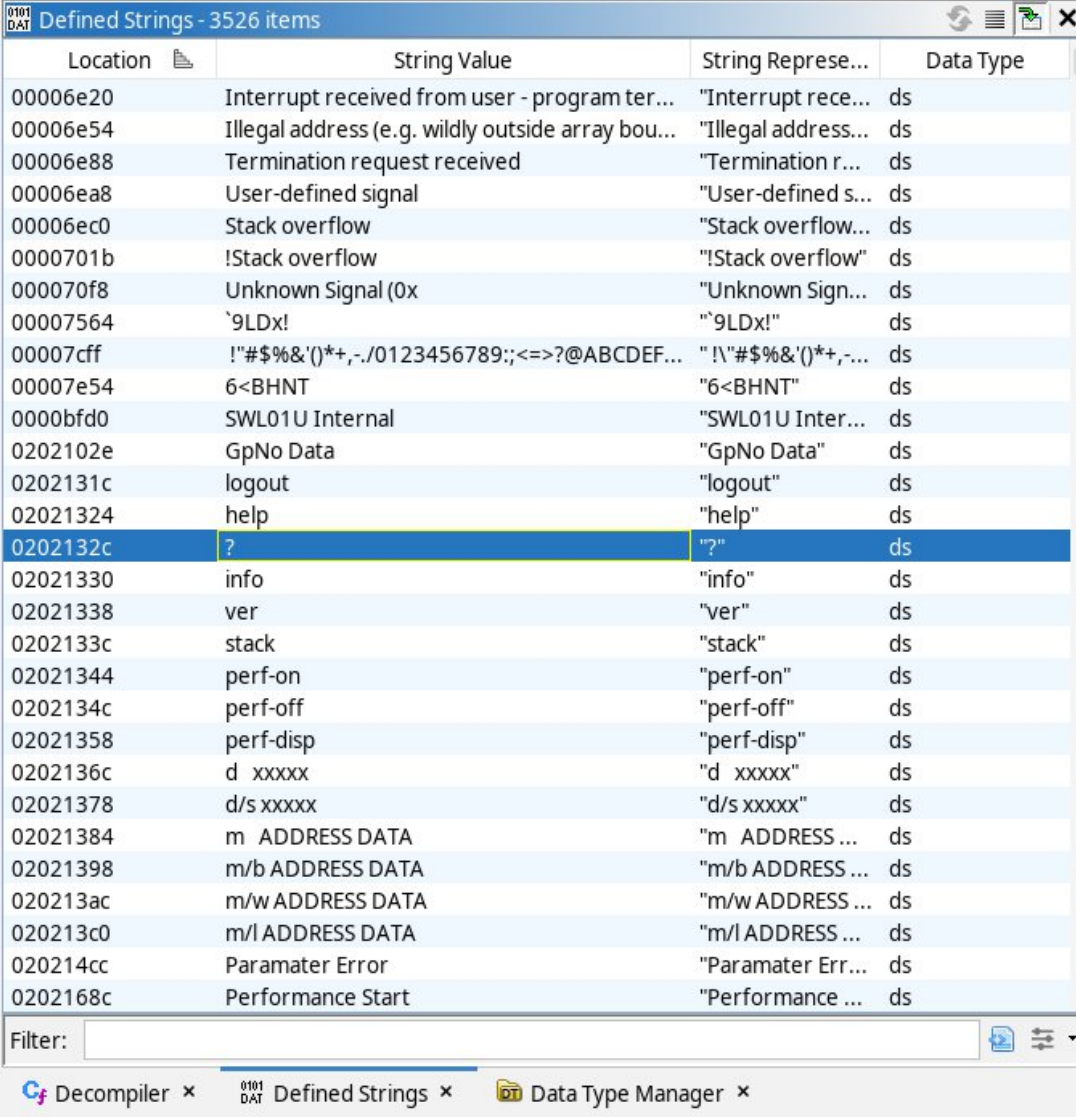
- The process is repetitive and boring to watch
- Let's look at one concrete subsystem

The shell



Suspicious strings

- Commands for some kind of shell?
 - help, ?
 - info
 - ver
 - perf-on, perf-off, perf-disp
- Some interesting commands
 - Memory reads?
 - d, d/s
 - Memory writes?
 - m, m/b, m/w, m/l
 - Remind me of GDB commands



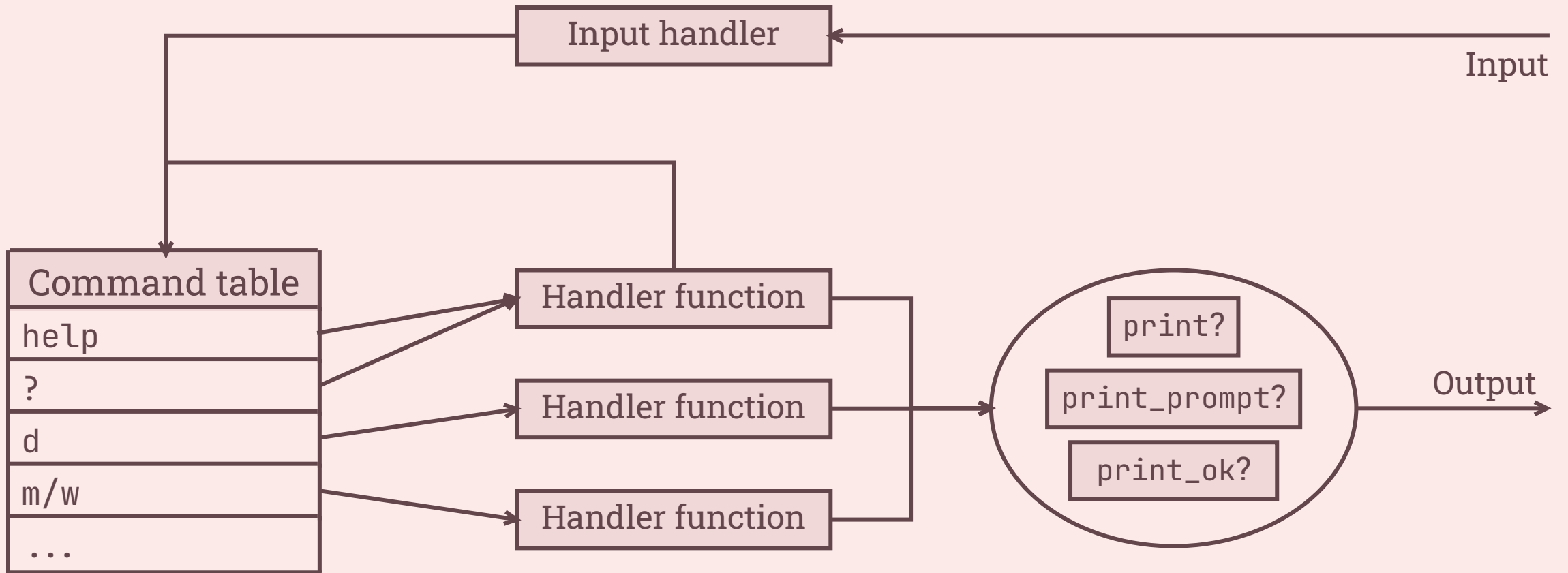
0101 DAT Defined Strings - 3526 items

Location	String Value	String Represe...	Data Type
00006e20	Interrupt received from user - program ter...	"Interrupt rece...	ds
00006e54	Illegal address (e.g. wildly outside array bou...	"Illegal address...	ds
00006e88	Termination request received	"Termination r...	ds
00006ea8	User-defined signal	"User-defined s...	ds
00006ec0	Stack overflow	"Stack overflow...	ds
0000701b	!Stack overflow	"!Stack overflow"	ds
000070f8	Unknown Signal (0x	"Unknown Sign...	ds
00007564	`9LDx!	"`9LDx!"	ds
00007cff	!"#\$%&'()*+,-./0123456789;<=>?@ABCDEF...	"!"#\$%&'()*+,-...	ds
00007e54	6<BHNT	"6<BHNT"	ds
0000bfd0	SWL01U Internal	"SWL01U Inter...	ds
0202102e	GpNo Data	"GpNo Data"	ds
0202131c	logout	"logout"	ds
02021324	help	"help"	ds
0202132c	?	"?"	ds
02021330	info	"info"	ds
02021338	ver	"ver"	ds
0202133c	stack	"stack"	ds
02021344	perf-on	"perf-on"	ds
0202134c	perf-off	"perf-off"	ds
02021358	perf-disp	"perf-disp"	ds
0202136c	d xxxxx	"d xxxxx"	ds
02021378	d/s xxxxx	"d/s xxxxx"	ds
02021384	m ADDRESS DATA	"m ADDRESS ...	ds
02021398	m/b ADDRESS DATA	"m/b ADDRESS ...	ds
020213ac	m/w ADDRESS DATA	"m/w ADDRESS ...	ds
020213c0	m/l ADDRESS DATA	"m/l ADDRESS ...	ds
020214cc	Paramater Error	"Paramater Err...	ds
0202168c	Performance Start	"Performance ...	ds

Filter:

Decompiler x 0101 DAT Defined Strings x Data Type Manager x

Shell architecture



Shell architecture

```
void shell_input_handler(char* command_input) {
    if (shell_login_state == 0) {
        if (shell_compare_command(command_input, "login") == 0) {
            shell_print("passwd?\r");
            shell_login_state = 1;
        }
    } else if (shell_login_state == 1) {
        if (shell_compare_command(command_input, "#0000") == 0) {
            shell_print("login OK\r");
            shell_login_state = 2;
        } else {
            shell_print("Passwd Error\r");
            shell_login_state = 0;
        }
    } else {
        // actually run the command
    }
}
```


Shell I/O

- UART?
 - Both hardware UARTs are unusable
 - 2 out of 2 receive pins not used as UART
 - 1 out of 2 transmit pins not used as UART
 - Software UART?
 - UART is ready to transmit bytes as-is
 - No heavy data manipulation required

Shell I/O

- JTAG?
 - ARM7's JTAG port features a "DCC"
 - Debug Communication Channel is bi-directional
 - DCC uses special instructions
- USB?
 - Could have a virtual network interface
 - Or a virtual serial port
 - Both should appear in USB descriptor

Shell I/O

```
$ lsusb -d 0499:1617 -v
Bus 001 Device 051: ID 0499:1617 Yamaha Corp. PSR-E353 digital keyboard
# ...
Device Descriptor:
# ...
bNumConfigurations      1
Configuration Descriptor:
# ...
bNumInterfaces          2
# ...
Interface Descriptor:
# ...
bInterfaceClass          1 Audio
bInterfaceSubClass       1 Control Device
# ...
Interface Descriptor:
# ...
bInterfaceClass          1 Audio
bInterfaceSubClass       3 MIDI Streaming
```

Shell I/O

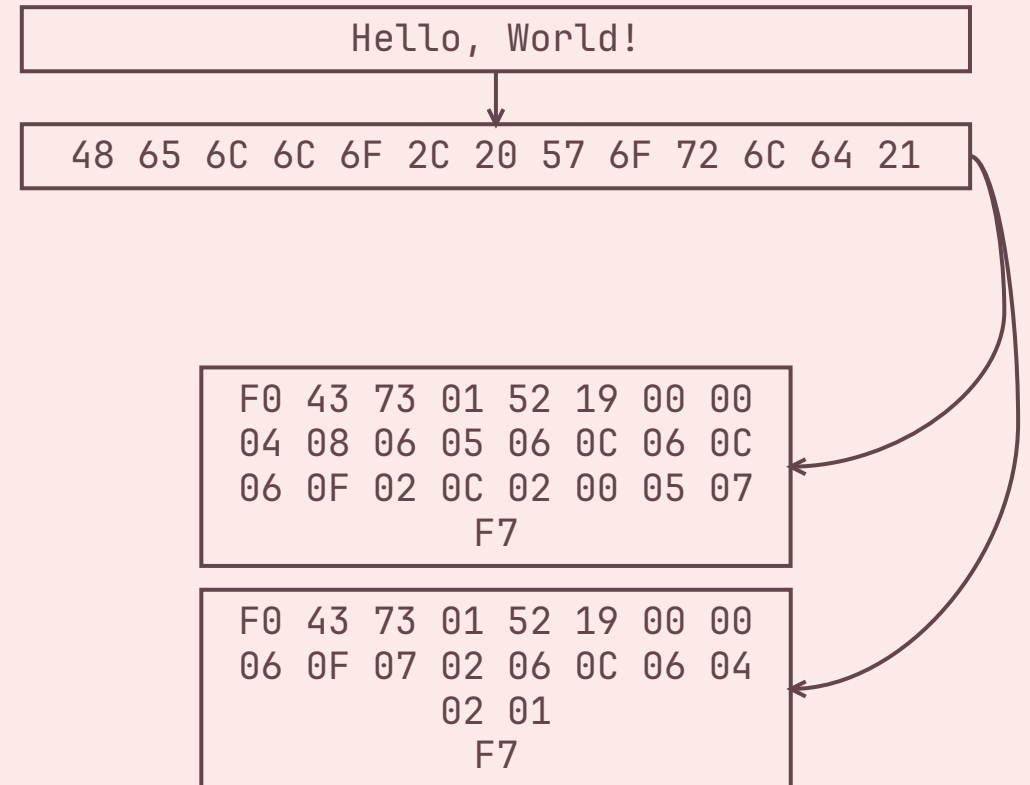
- Probably not hardware UART – the interfaces are taken
 - Software UART still on the table
- Probably not USB – only has a MIDI descriptor
- Maybe JTAG DCC?
- Let's look at `shell_print` to find out

Shell I/O

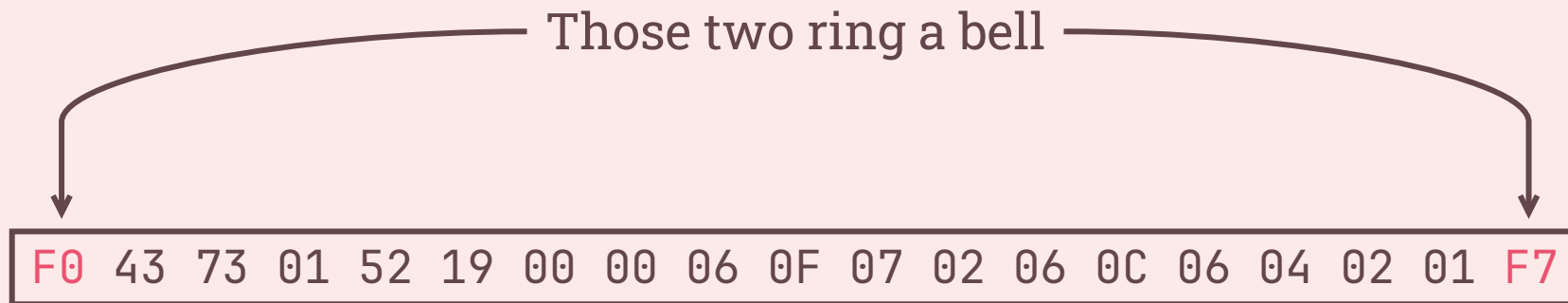
```
void shell_print(char* data) {
    memcpy(send_buf, const_buf, 8);
    send_buf[24] = 0xf7;

    // process data in 8-byte blocks first
    for(size_t i = strlen(data); i ≥ 8; i -= 8) {
        char ch = *(data++);
        for(int j = 0; j < 8; j++) {
            send_buf[8 + (j * 2)] = ch >> 4;
            send_buf[8 + (j * 2) + 1] = ch & 0xf;
        }
        pass_on_to_next_stage_of_printing(send_buf, 25);
    }

    // process the tail
    size_t tail_size = 8;
    for(int i = 0; *data; i++) {
        char ch = *(data++);
        send_buf[8 + (i * 2)] = ch >> 4;
        send_buf[8 + (i * 2) + 1] = ch & 0xf;
        tail_size += 2;
    }
    send_buf[tail_size] = 0xf7;
    tail_size++;
    pass_on_to_next_stage_of_printing(send_buf, tail_size);
}
```



Shell I/O



SysEx

- That's a so-called System Exclusive message!
- MIDI having SysEx is like Python having assembly snippets

"11110000 [binary] – System Exclusive"

"This message type allows manufacturers to create their own messages <...>."

"The Manufacturer's ID (assigned by MMA <...>) is either 1 byte or 3 bytes"

MIDI Manufacturers Association,
Summary of MIDI 1.0 Messages
<https://midi.org/spec-detail>

Start of SysEx

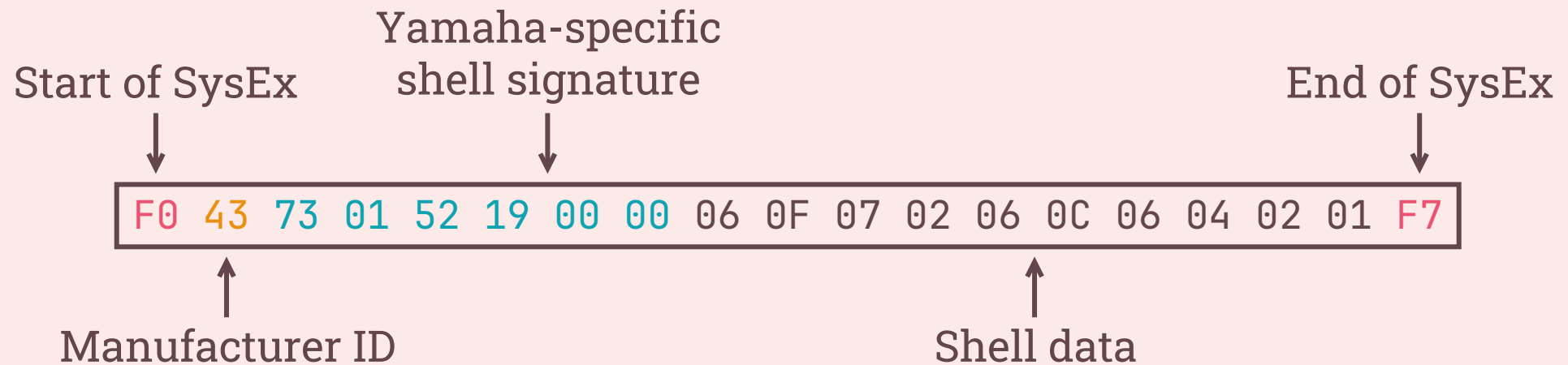
End of SysEx

F0 43 73 01 52 19 00 00 06 0F 07 02 06 0C 06 04 02 01 F7

Manufacturer ID

SysEx

“43H – Yamaha Corporation”
MIDI Manufacturers Association,
SysEx ID table
<https://midi.org/sysexidtable>



Accessing the shell

- Let's write a Python script that talks to the device in this format
- Possibility of failure
 - Format of incoming messages could differ
 - Shell might be disabled permanently
 - ...or require enabling
- Worth a try though!
- [Video: shell access demo]

Exploitation



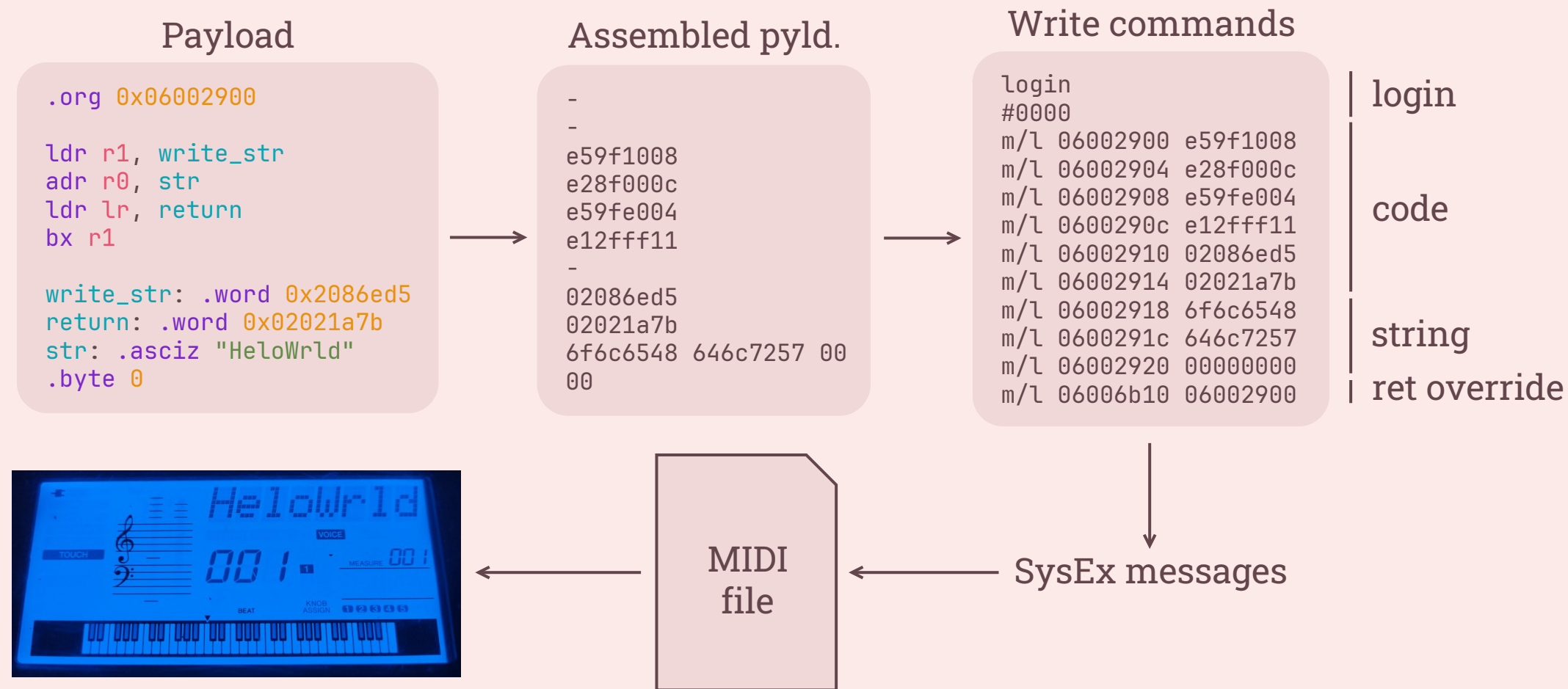
Exploitation

- Memory write commands!
- For real?
- We don't need JTAG. We just need a malicious MIDI file.
 - Write executable payload into memory
 - Overwrite return address on stack
 - In payload, do malicious things
 - In payload, jump to original return address

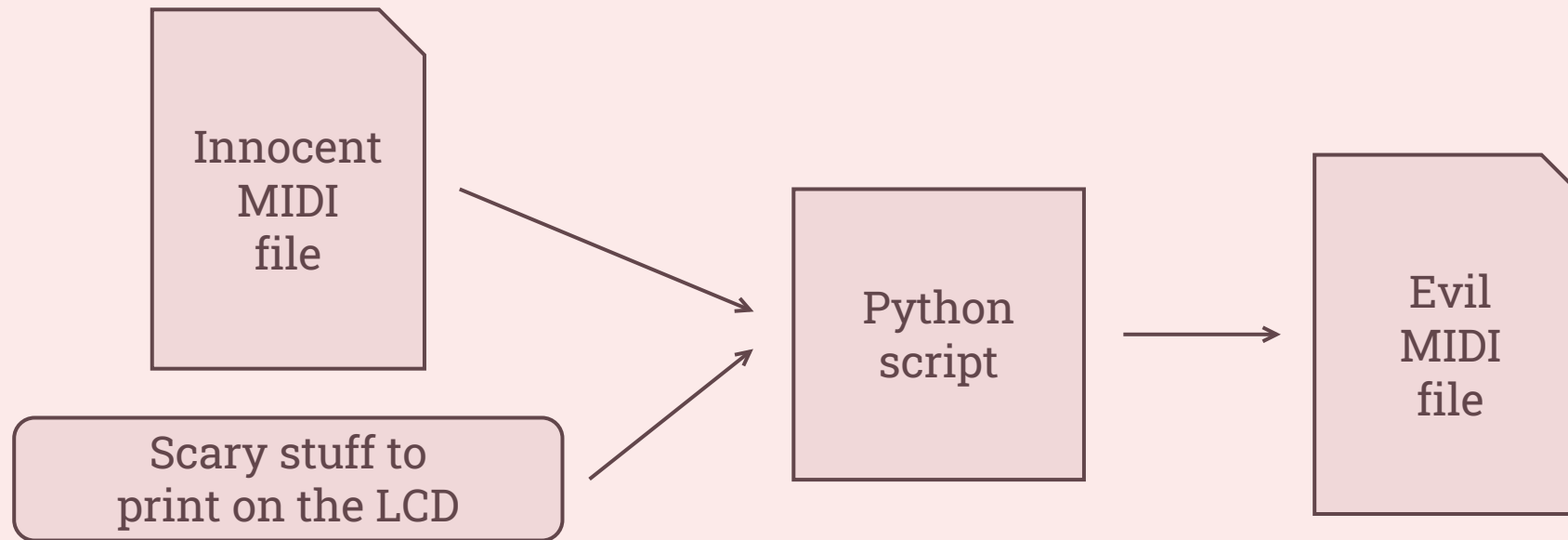
```
# synth shell
> m/l 06001234 ab0ba123
>

# gdb
(gdb) x/1xw 0x06001234
0x06001234:      0xab0ba123
(gdb)
```

Exploitation



Exploitation



[Video: Scary toccata]

Bad Apple



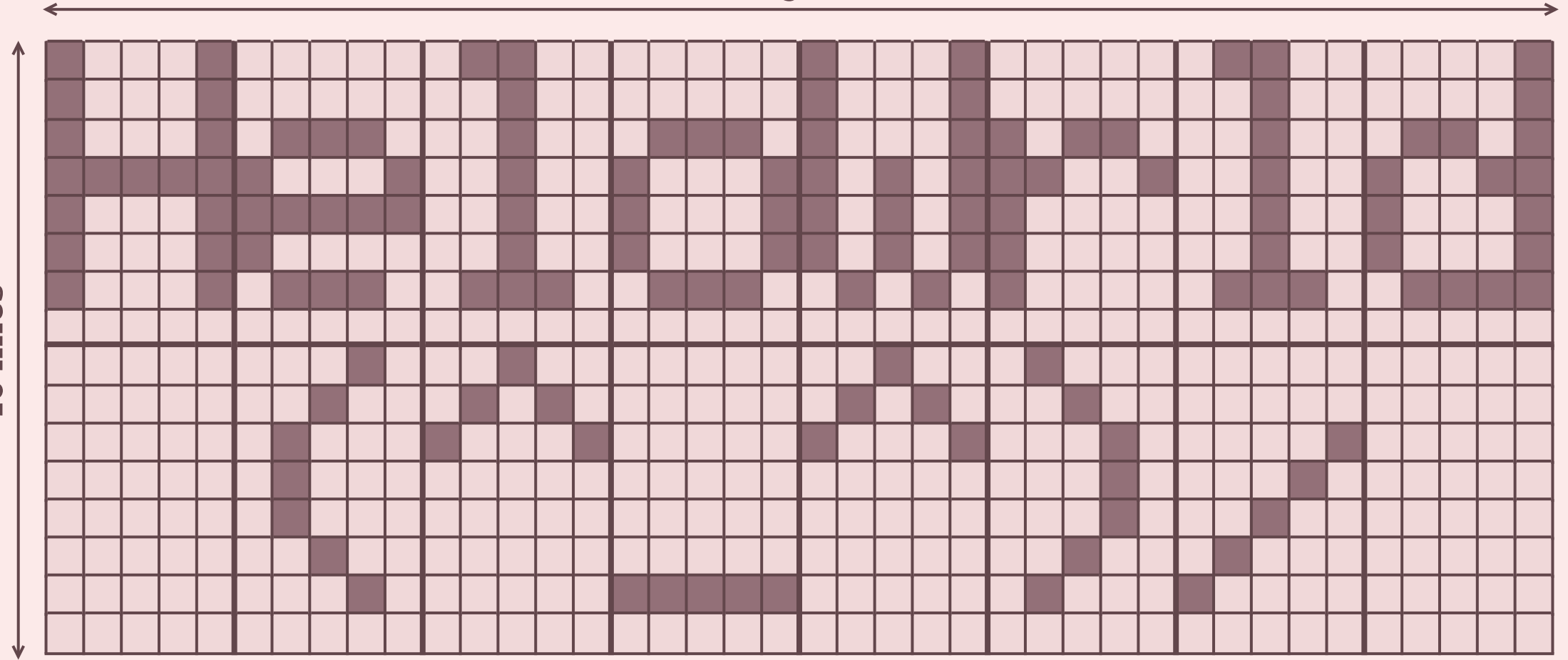
Displaying video

- The LCD controller is only designed to display text
 - 2 lines of 8 characters
 - Characters are 5 lines of 8 pixels
- No pictures, no video
- How is the synth displaying non-textual graphics?
- How are we going to play video?

LCD that our controller expects

40 segments

16 lines



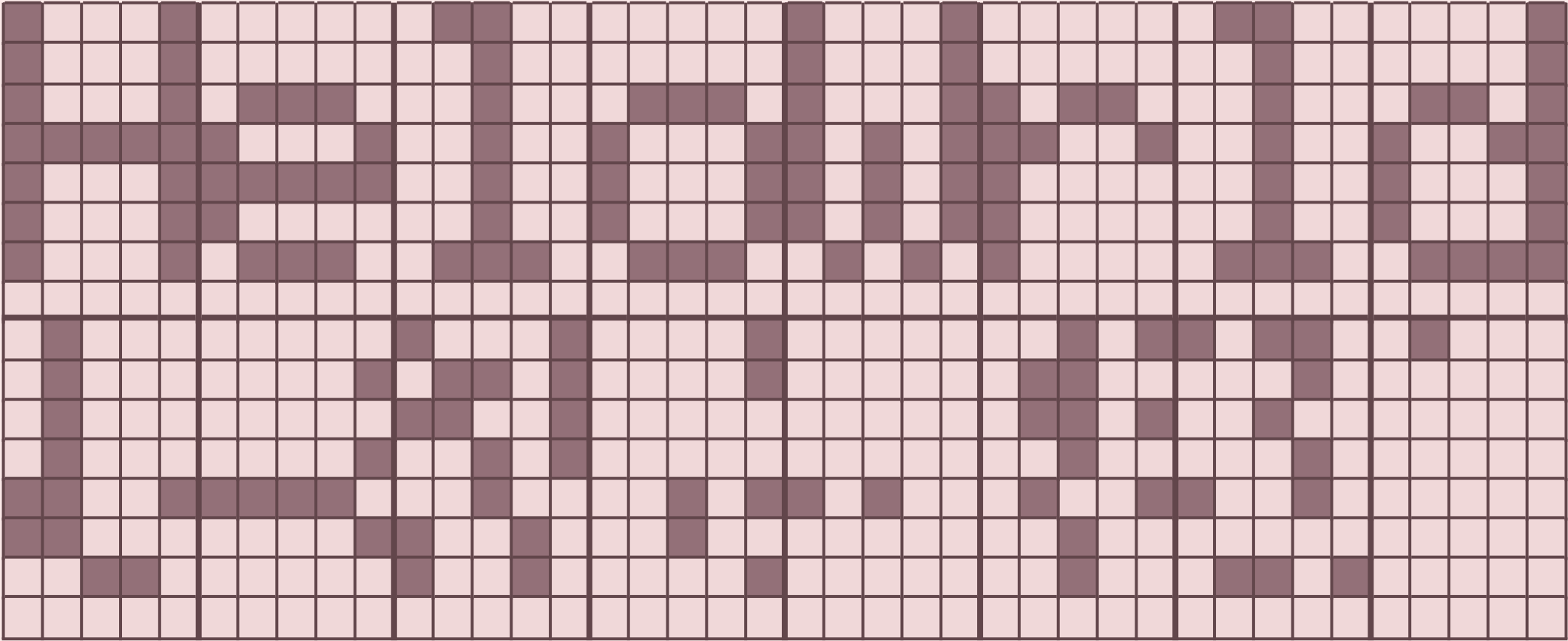
Our LCD

40 segments

16 lines

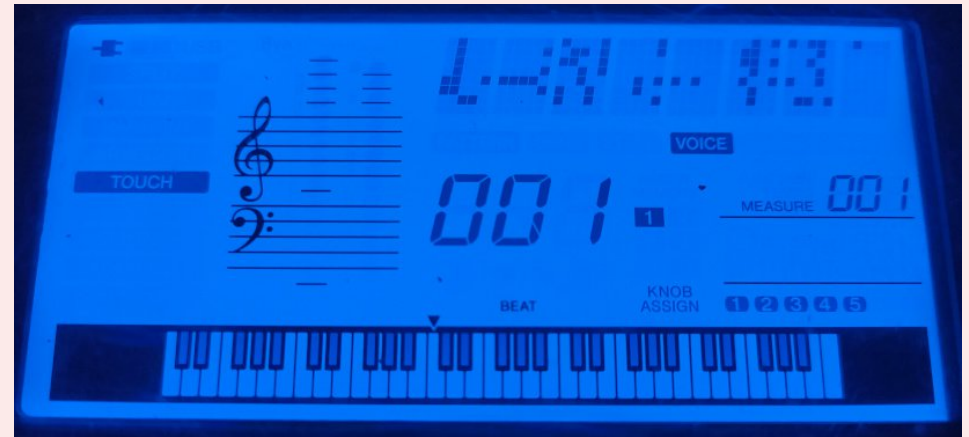
Text

Other
stuff

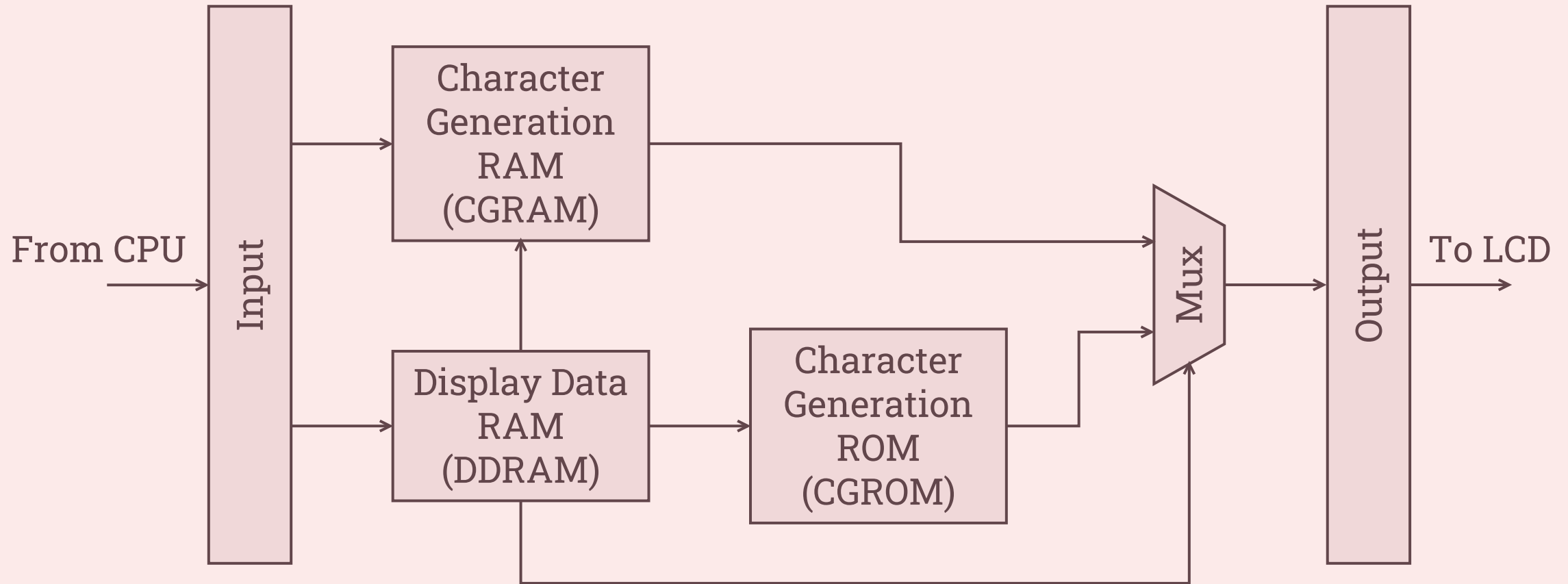


Our LCD

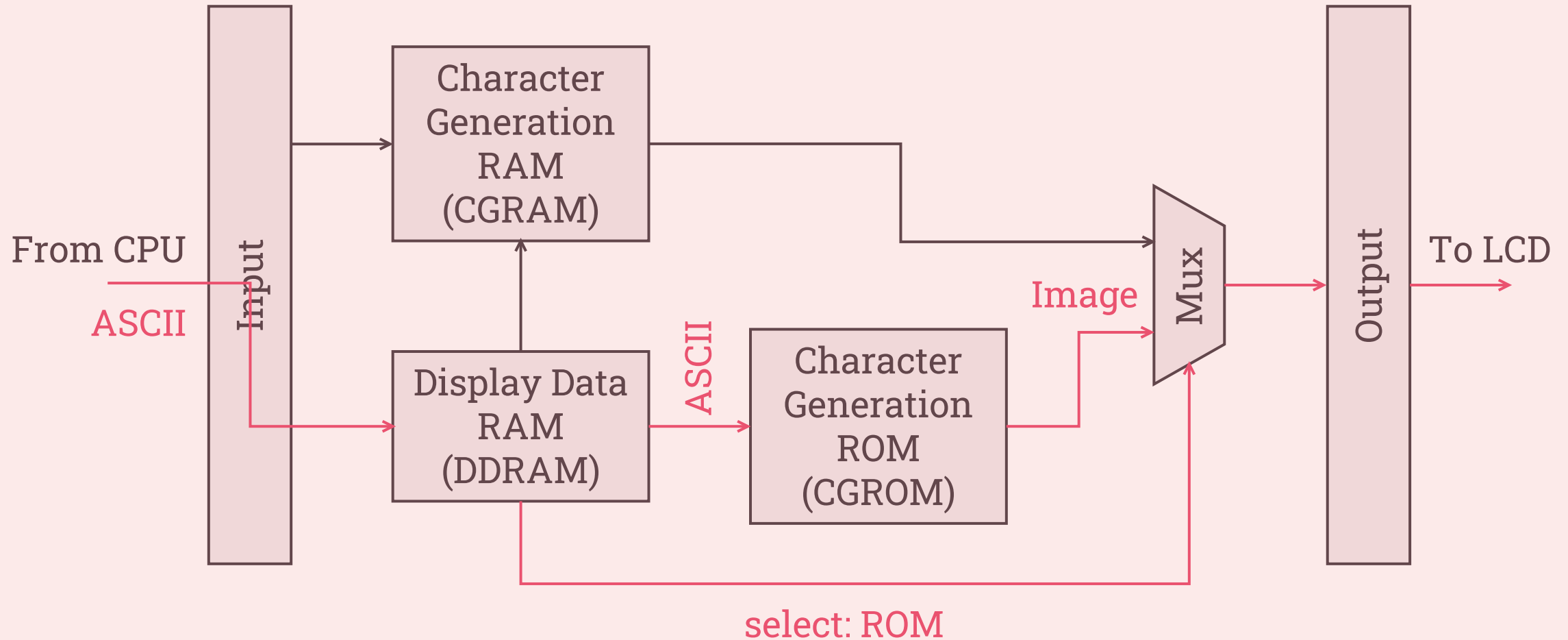
- Top row of text is wired to the dot matrix area
 - ...as it should
- Bottom row of text is wired to all the other segments
- CPU somehow displays custom data in false bottom row
 - ...this is what that data looks like when rendered in the top row
- How? And can we do the same?



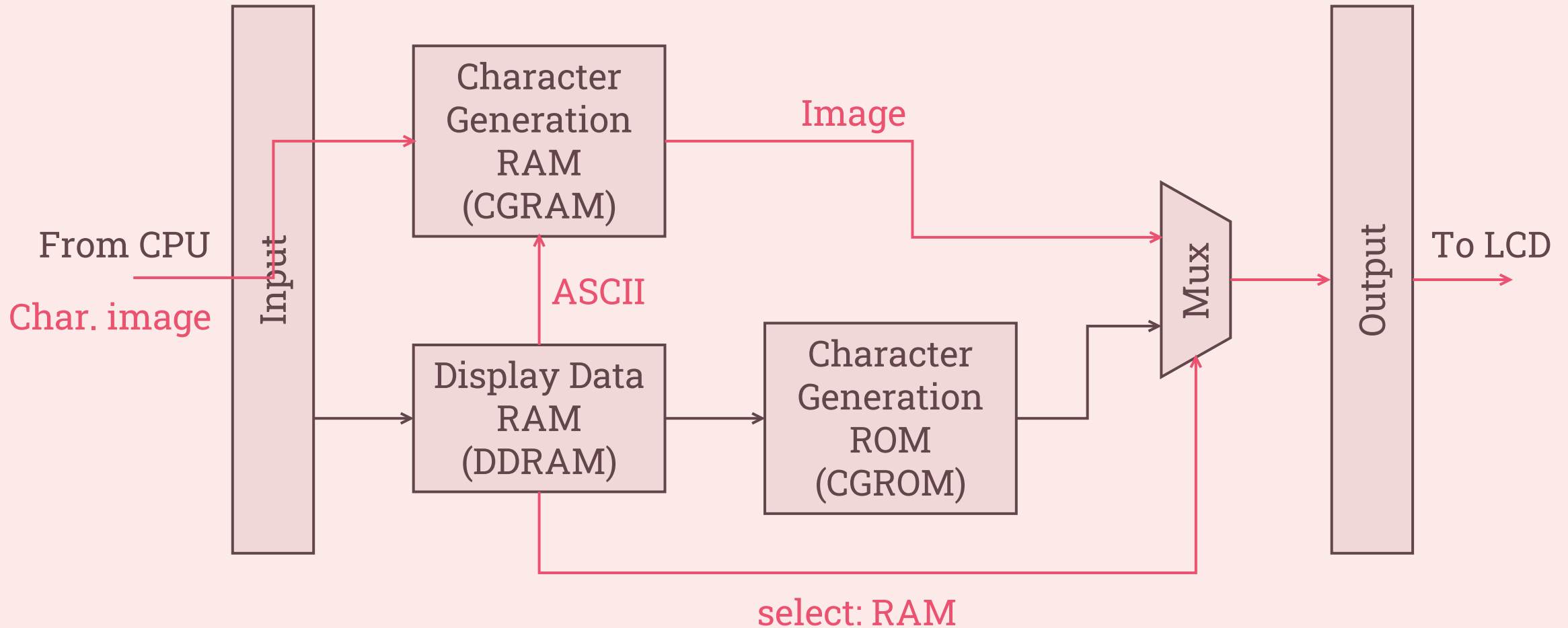
LCD controller



LCD controller



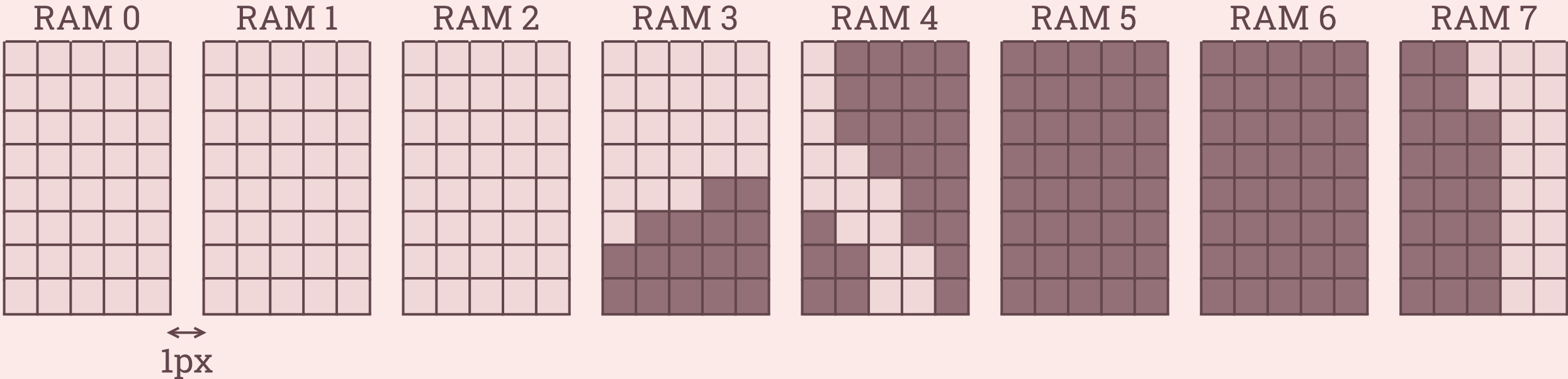
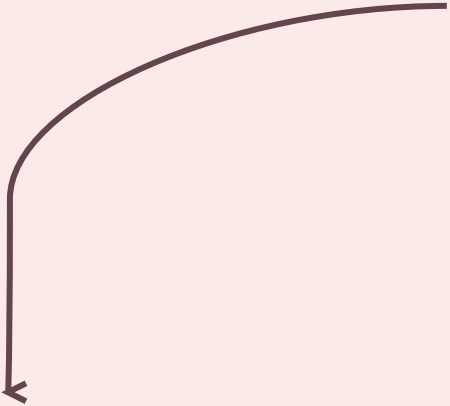
Custom characters



Custom characters

- The CGRAM can fit 8 custom characters
- They are accessed by character numbers 0 through 7 and 8 through 15

Displaying images



The synth fights back

- When I load my images into CGRAM, the synth overwrites them with its junk
- We need to disable that part of the firmware
- I don't want to change what's written on the flash
- There's a way!

An unknown RTOS



An unknown RTOS

- Remember how there's also a ROM?
 - ROM is really Read-Only, and it's embedded in the CPU
 - Flash is programmable, for updates
- ROM contains some very basic code abstractions over the chip and parts of a Real-Time Operating System
- ROM and flash are loosely coupled

An unknown RTOS

- Sometimes, the flash calls into the ROM
 - It knows where to call because the ROM never changes
- Sometimes, the ROM calls into the flash to handle an event
 - It doesn't know where to call because the flash could be updated
- There's a linked series of task tables in flash that facilitate this linkage

RTOS hijacking

- We can make a copy of the task tables, but in RAM
- Modify them as we wish
 - Disable LCD updates
 - Install our own shell input handler for speed
- Then tell the ROM that the tables are located at this new address
- Result? Video playback at 30fps
- [Video: Bad Apple demo from one of the first slides]

Addendum



Vulnerable products

- Exploit successfully reproduced on these Yamaha products:
 - PSR-E433 (ARM7, SWL01U) - 2012
 - PSR-E453/E473 (ARM Cortex, SWX03) - 2016, 2022
 - DGX-660 (SH-2, SWX08) - 2016
 - PSS-A50/E30/F50 (ARM, YMW830-V)
 - P-125 (ARM Cortex, SWX09) - 2018
- Possibly vulnerable, not tested:
 - Digital Finger Drum series, specifically FGDP-30/50
- ...all with #0000 shell passwords
- Ongoing thread: github.com/portasynthinca3/swl01u/issues/1
 - Shoutout to Marisa-Chan, Crawlerop, YoYo1846 and xintrea

DSP summary

- The DSP is independent of the ARM core
- DSP MMIO region starts at address 0xfe000000
 - Global configuration area: 0xfe000000 - 0xfe0003ff
 - Voice parameter area: 0xfe000400 - 0xfe0007ff
- 32 voices
- 16 16-bit parameters per voice
 - Same parameters for different voices arranged in one area
 - $\text{par_area_start} = 0\text{xfe}000400 + (0\text{x}40 * \text{par_no})$
 - $\text{par_location} = \text{par_area_start} + (2 * \text{voice_no})$
- All registers and voice parameters are written byte-wise in big endian:
MSB to LSB

DSP summary

- Known voice parameters (0-based indexing):
 - Index 3: ADLR (Amplitude Difference and Level Register)
 - High byte: dual-channel attenuation
 - Low byte: separate channel attenuation (one nibble for each)
 - Index 4: ARAR (Attack Rate Register)
 - Top bit: attack enabled; bottom 15 bits: attack speed
 - Not sure about the attack speed format
 - Index 5: LSTR (Length and Start of Taper-off Register)
 - Write of any value starts the decay
 - Not sure about the decay speed format
 - Index 8: STCR (Signal Tone Control Register)
 - Top nibble: 2's complement octave selection
 - Bottom 12 bits: unsigned fine adjust
 - $f = (440\text{Hz} * 2^{\text{octave}}) * (\text{fine} / 0x400)$

DSP summary

- Known global registers:
 - 0xfe000008: STAR (Start Register), 5-byte, big endian, bitwise write
 - Each of the top 32 bits signifies a channel which has to be started
 - The bottom 8 bits must be set to 0x80 to active the start

References

- <https://sandsoftwaresound.net/swl-micro-architecture/>
- <https://midi.org/spec-detail>
- <https://midi.org/sysexidtable>
- <https://developer.arm.com/documentation/ddi0084/f>

That's it!



Thank you <3